

1. Локализация настроек

CREATE TABLESPACE WITH ALTER TABLESPACE SET	Табличное пространство Только для параметров: <code>seq_page_cost</code>, <code>random_page_cost</code>, <code>effective_io_concurrency</code>, <code>maintenance_io_concurrency</code>	<code>CREATE TABLESPACE ts1</code> <code>LOCATION '/data/ts1'</code> <code>WITH (random_page_cost = 1.4);</code> <code>ALTER TABLESPACE ts1</code> <code>SET maintenance_io_concurrency = 16;</code>
ALTER DATABASE SET	База данных	<code>ALTER DATABASE planner</code> <code>SET work_mem = '64MB';</code>
ALTER ROLE SET	Роль	<code>ALTER ROLE analyst</code> <code>SET work_mem = '256MB';</code>
ALTER ROLE IN DATABASE SET	Роль и база данных В случае конфликта параметры на уровне роли переопределяют параметры на уровне БД.	<code>ALTER ROLE analyst</code> <code>IN DATABASE dwh</code> <code>SET work_mem = '256MB';</code>
ALTER ROUTINE SET (ALTER FUNCTION SET ALTER PROCEDURE SET)	Подпрограмма	<code>ALTER FUNCTION get_flights</code> <code>SET work_mem = '128MB';</code>
SET [LOCAL]	Сеанс / транзакция	<code>SET work_mem = '64MB';</code> <code>SET LOCAL work_mem = '64MB';</code>

2. Модель стоимости ввода-вывода (I/O Cost Model)

seq_page_cost	1.0	Стоимость последовательного чтения страницы с диска. Эталон: Все остальные стоимости настраиваются относительно <code>seq_page_cost</code> (во сколько раз операции дороже/дешевле последовательного чтения).
random_page_cost	4.0 ↓	Стоимость произвольного чтения страницы с диска. При уменьшении относительно <code>seq_page_cost</code> планировщик чаще выбирает индексное сканирование. Значение 4.0 подходит для традиционных HDD дисков, значения 1.1-1.2 – для современных SSD.
effective_io_concurrency	1	Допустимое число параллельных операций I/O. Влияет на предварительное чтение страниц (prefetching) при Bitmap Scan.
maintenance_io_concurrency	10	Аналог <code>effective_io_concurrency</code> , но для операций обслуживания.

Параметры I/O могут быть заданы на уровне TABLESPACE.

Параметры `effective_io_concurrency`, `maintenance_io_concurrency` в Windows и MacOS равны **0** (ОС сама управляет стратегией кэширования и предварительного чтения). На Linux для управления предварительным чтением используется `posix_fadvise()`.

3. Модель стоимости CPU (CPU Cost Model)

cpu_tuple_cost	0.01	Стоимость обработки одной табличной строки при выполнении запроса.
cpu_index_tuple_cost	0.005	Стоимость обработки одной записи индекса при сканировании индекса.
cpu_operator_cost	0.0025	Стоимость элементарной операции обработки одной строки оператором или функцией. Операции: арифметические, битовые, логические, операции сравнения, вызовы встроенных функций (<code>lower()</code>, <code>abs()</code>, <code>extract()</code> и др.), приведение типов.

4. Настройки памяти

shared_buffers	128MB↑	Объем памяти, который будет использовать сервер баз данных для буферов в разделяемой памяти (задается в единицах объема информации). Если задается без единиц объема информации, считается заданным в страницах, т.е., если страница 8 kB, то '128MB' = 16384. Часто разумным начальным значением <code>shared_buffers</code> будет 25% от объема памяти.
temp_buffers	8MB	Максимальный объем памяти, выделяемой для буферов временных таблиц (TEMP TABLE) в каждом сеансе.
work_mem	4MB ↑	Максимальный объем памяти, который будет использоваться сеансом для одной внутренней операции (сортировка, хеш-таблицы, построение битовой карты и пр.) для конкретного узла плана.
hash_mem_multiplier	2.0	Коэффициент для определения максимального объема памяти, который может выделяться для операций с хешированием. Итоговый объем: <code>work_mem x hash_mem_multiplier</code>. Доступен с версии PostgreSQL 13 (в версиях 13 и 14 значение по умолчанию – 1.0)
maintenance_work_mem	64MB	Максимальный объем памяти для операций обслуживания БД - VACUUM, CREATE INDEX, ALTER TABLE ADD FOREIGN KEY.
effective_cache_size	4GB	Эффективный размер файлового кэша ОС, доступный для одного запроса. Параметр служит только для оценки и не приводит к реальному выделению памяти. Чем больше <code>effective_cache_size</code> относительно размера таблицы, тем дешевле считается Seq Scan (но! учитываются также селективность, стоимость index scan и т.д.). Предполагается, что значение <code>effective_cache_size</code> должно соответствовать общему объему памяти, доступному для кэширования (включая и <code>shared_buffers</code> , и кэш ОС).

5. Настройки временных файлов

temp_file_limit	-1	Максимальный объем дискового пространства, который может использовать один процесс для временных файлов. Запись на диск происходит при недостаточном объеме <code>work_mem</code> для заданной операции. Параметр задается в кВ (-1 – ограничение отсутствует). - Операции: сортировка, хеширование, сохранение удерживаемого курсора. Не связан с файлами явно создаваемых временных таблиц. - Все временные файлы создаются в табличном пространстве по умолчанию, установленном для текущей БД (например, <code>PGDATA/base/dbid/pgsql_tmp</code>).
temp tablespaces	''	Задаёт табличные пространства, в которых будут создаваться временные объекты (для сортировки, хеширования), в т.ч. временные таблицы и индексы временных таблиц. Значение по умолчанию – пустая строка: все временные объекты создаются в табличном пространстве по умолчанию, установленном для текущей БД. Значение параметра: имя или список имён табличных пространств.
log_temp_files	-1	Управляет регистрацией в журнале сообщений имён и размеров временных файлов. При значении 0 записывается информация обо всех файлах, а при положительном – о файлах, размер которых не меньше заданной величины (в кВ). Значение по умолчанию: -1, запись отключена.

6. Настройки параллельного выполнения

parallel_setup_cost	1000	Стоимость запуска параллельных рабочих процессов.
parallel_tuple_cost	0.1	Стоимость передачи одного кортежа от параллельного рабочего процесса другому процессу.
parallel_leader_participation	on	Позволяет ведущему процессу выполнять план запроса ниже узлов Gather и Gather Merge, не ожидая рабочие процессы.
parallel_workers		Число рабочих процессов, которые должны задействоваться при параллельном сканировании таблицы. Не конфигурационный параметр! Параметр таблицы (<code>CREATE / ALTER TABLE</code>). <code>ALTER TABLE SET</code> требует блокировку <code>SHARE UPDATE EXCLUSIVE</code> .
max_worker_processes	8	Число одновременно выполняющихся рабочих процессов. Изменение через <code>postgresql.conf</code> / <code>ALTER SYSTEM</code> , требует перезагрузки PostgreSQL. Включает: - Параллельные процессы для запросов (из <code>max_parallel_workers</code>), - Процессы логической репликации, - Пользовательские фоновые процессы (расширения), - Некоторые системные процессы (<code>autovacuum worker</code>, при выполнении команды <code>CREATE INDEX CONCURRENTLY</code>)
max_parallel_workers	8	Число одновременно выполняющихся рабочих процессов, занимающихся параллельными планами. $\text{max_parallel_workers} \leq \text{max_worker_processes}$ Значение <code>max_parallel_workers</code> , превышающее <code>max_worker_processes</code> , не будет действовать, так как параллельные рабочие процессы берутся из пула рабочих процессов, ограничиваемого этим параметром. Если нет свободных рабочих процессов – немедленное последовательное выполнение.
max_parallel_maintenance_workers	2	Максимальное количество рабочих процессов для одной служебной команды. $\text{max_parallel_maintenance_workers} \leq \text{max_parallel_workers}$ Параллельные процессы могут использовать команды <code>CREATE INDEX</code> при построении индекса В-дерева (с версии PostgreSQL 11, с PostgreSQL 17 – также <code>BRIN</code> , с PostgreSQL 18 – также <code>GIN</code>) и <code>VACUUM</code> без указания <code>FULL</code> (с версии PostgreSQL 13). Значение 0 отключает параллельное выполнение служебных команд.
max_parallel_workers_per_gather	2	Максимальное количество рабочих процессов, которые могут быть запущены для одной операции Gather в параллельном запросе. Значение 0 отключает параллельное выполнение запросов.

Parallel Seq Scan

min_parallel_table_scan_size	8MB	Минимальный размер таблицы для рассмотрения параллельного последовательного сканирования.
Расчет числа рабочих процессов Но не более, чем значение <code>max_parallel_workers_per_gather</code>		
0	Если размер таблицы меньше значения параметра <code>min_parallel_table_scan_size</code> (8 MB).	
Фикс.	Если для таблицы указан параметр <code>parallel_workers</code> . <code>CREATE TABLE meetings WITH (parallel_workers = 8);</code> <code>ALTER TABLE bookings (SET parallel_workers = 4);</code>	
Вычисляется по формуле	$1 + \log_3 \frac{\text{table size}}{\text{min_parallel_table_scan_size}}$	Пример. Если размер таблицы 240 MB: $1 + \log_3 \frac{240 \text{ Mb}}{8 \text{ Mb}} \approx 4$

Parallel Index Scan, Parallel Index Only Scan, Parallel Bitmap Heap Scan

min_parallel_index_scan_size	512kB	Минимальный размер выборки для рассмотрения параллельного сканирования по индексу.
Расчет числа рабочих процессов Но не более, чем значение max_parallel_workers_per_gather		
0	Если размер выборки меньше значения параметра min_parallel_index_scan_size (512 kB).	
Фикс.	Если для таблицы указан параметр parallel_workers .	
Вычисляется по формуле	$1 + \log_3 \frac{result\ set\ size}{min_parallel_index_scan_size}$	Пример. Если размер выборки 5 MB: $1 + \log_3 \frac{5120\text{KB}}{512\text{KB}} \approx 3$

7. Настройки курсоров

cursor_tuple_fraction	0.1	Задаёт долю строк, которую предполагается извлечь через курсор. - Малые значения → приоритет быстрого получения первых строк 1.0 → курсор планируется как обычный запрос
Минимизируется значение: $cost1 + (cost2 - cost1) \times cursor_tuple_fraction$ Где: - $cost1$ = стартовая стоимость плана (startup cost) - $cost2$ = полная стоимость плана (total cost)		

8. Настройки подготовленных операторов

- Custom plan.** Строится заново с учётом параметров → высокая стоимость планирования.
- Generic plan.** Один план на все значения параметров → может быть неоптимален.

PostgreSQL обычно анализирует первые ~5 выполнений, но решение принимается по стоимости, а не по счётчику (если generic дешевле → используется generic plan, а если custom дешевле → PostgreSQL продолжает строить custom plan каждый раз).

plan_cache_mode	auto	- auto – автоматический выбор (по умолчанию), - force_generic_plan – всегда использовать общий план (generic), - force_custom_plan – всегда перестраивать план (custom). Значение этого параметра учитывается при выполнении плана, а не при построении.
---------------------------------	------	--

9. Настройки функций**Категории изменчивости**

VOLATILE	По умолч.	Функция может возвращать разные результаты даже при одинаковых аргументах в рамках одного запроса (например, random(), nextval()). Оптимизатор будет вызывать её ровно столько раз, сколько она встречается в плане выполнения.
STABLE		Функция гарантированно возвращает одинаковый результат, получая одинаковые аргументы, для всех строк в одном операторе. Эта характеристика позволяет оптимизатору заменить множество вызовов этой функции одним. ALTER FUNCTION rnd() STABLE;
IMMUTABLE		Функция всегда возвращает одинаковый результат для одинаковых аргументов (как математическая функция sin(x)). Оптимизатор может вычислить её значение один раз и подставить константу. ALTER FUNCTION имя_функции(...) VOLATILE STABLE IMMUTABLE;

Inline (подстановка кода функции в запрос)

Скалярные функции	Табличные функции (RETURNS SETOF / TABLE)
- Язык SQL. - Является SECURITY INVOKER (режим по умолчанию). - Тело состоит из одного простого выражения SELECT. Тело не содержит вызовов агрегатных или оконных функций, подзапросов, CTE, предложения FROM, а также операторов GROUP BY, HAVING, ORDER BY, DISTINCT, LIMIT, OFFSET, UNION, INTERSECT, EXCEPT. - Не должна возвращать составной тип (RETURNS RECORD). - В определении функции отсутствуют предложения SET. - Если функция IMMUTABLE, то выражение не должно вызывать какие-либо не-IMMUTABLE функции или операторы. Если функция STABLE, то выражение не должно вызывать какие-либо VOLATILE-функции или операторы.	- Язык SQL. - Является SECURITY INVOKER (режим по умолчанию). - Тело функции должно состоять из одного оператора SELECT, даже после применения любых правил трансформации. - В определении функции отсутствуют предложения SET. - Функция объявлена как STABLE или IMMUTABLE. - Не объявлена как STRICT. - Вызов функции не указывает ORDINALITY или несколько функций в ROWS FROM. - Ни один из фактических аргументов не содержит volatile-выражений или подзапросов. Возможна материализация функции (узел Function Scan) при вызове через SELECT FROM, а также «ленивая» генерация (узел ProjectSet) при вызове через SELECT.

Настройки распараллеливания функций (PARALLEL)

UNSAFE	По умолч.	Запрет параллельных планов, если в запросе есть вызов UNSAFE функции.
RESTRICTED		Разрешаются параллельные планы для других функций в запросе, кроме RESTRICTED функции.
SAFE		Разрешены параллельные планы.

Настройки функций

COST	100	Стоимость возврата одного значения в единицах <code>cpu_operator_cost</code> . Если функция возвращает множество (RETURNS SETOF / TABLE), это число задаёт стоимость возврата одной строки. Стоимость выполнения функции считается постоянной. Функции на С получают оценку 1.
ROWS	1000	Приблизительное число строк, которое будет ожидать планировщик на выходе этой функции. Установить это значение можно только, если функция возвращает множество (RETURNS SETOF / TABLE).
SUPPORT	<code>support_function</code>	Вспомогательная функция, которая вычисляет переменную оценку стоимости, в случае, когда алгоритм работы целевой функции значительно меняется в зависимости от её аргументов. Вспомогательные функции должны быть написаны на С. Многие встроенные функции содержат вспомогательные.
CREATE FUNCTION ... COST стоимость ROWS строки ALTER FUNCTION ... COST стоимость ROWS строки В общем случае для пользовательских функций, возвращающих множество, «начальная стоимость..полная стоимость» в плане равны: $COST * cpu_operator_cost .. (COST * cpu_operator_cost) + (ROWS * cpu_tuple_cost)$		

- Просмотреть соответствующие настройки можно в каталоге [pg_proc](#), столбцы:
 - `procost` (стоимость), `prorows` (число строк), `prosupport` (вспомогательная функция),
 - `proparallel` (настройка распараллеливания), `provolatile` (категория изменчивости),
 - `proretset` (признак табличной функции, если установлен, то `procost` – стоимость выдачи одной строки).

10. Управление JOIN и FROM

join collapse limit	8	Максимальное количество элементов в списке FROM .. JOIN, которое планировщик может переставлять местами для поиска наилучшего плана. В случае, если количество элементов в списке FROM .. JOIN превышает значение join collapse limit , то планировщик переходит к поэтапной оптимизации. Он разбивает список на группы (по <code>join_collapse_limit</code> элементов каждая или менее), определяет наиболее эффективный способ соединения внутри каждой группы, а затем рассматривает полученный результат уже как единый логический элемент (промежуточный набор строк) для соединения на следующем уровне. Для запросов через запятую (SQL89), т.е. <code>FROM t1, t2, t3...</code> этот параметр не действует. Планировщик перебирает все возможные способы соединений таблиц из списка без какой-либо дополнительной обработки (если не работает генетический алгоритм GEQO).
from collapse limit	8	Максимальное число вложенных элементов в списке FROM (SELECT FROM .. SELECT), до которого планировщик будет раскрывать вложенные запросы во внешний запрос. Аналогично параметру join collapse limit планировщик не будет пытаться рассмотреть все возможные комбинации разом, а будет оптимизировать запрос по частям.

- Обычно `join_collapse_limit` устанавливается равным `from_collapse_limit` (чтобы явные соединения и подзапросы обрабатывались одинаково) или 1 (значение 1 запрещает переупорядочивание явных JOIN).
- Эффект подобной оптимизации рекомендуется подтверждать экспериментально.

11. Отключение механизмов планировщика

По умолчанию эти параметры конфигурации имеют значение **on** (вкл.)

enable seqscan , enable indexscan , enable indexonlyscan , enable bitmapscan	Отключение определенных методов доступа: последовательное сканирование, сканирование индекса, сканирование только индекса, сканирование по битовой карте. Чтобы использовать планы <code>Index Only Scan</code>, также должен быть включён параметр <code>enable_indexscan</code>.
enable nestloop , enable hashjoin , enable mergejoin	Отключение определенных методов соединений: соединение вложенным циклом, соединение хэшированием, соединение слиянием.
enable sort , enable incremental sort , enable presorted aggregate	Отключение возможностей, связанных с сортировкой.
enable hashagg , enable material , enable memoize	Отключение определенных операций, связанных с агрегированием по хешу, материализацией и мемоизацией (enable memoize доступен в PostgreSQL 14).
enable parallel hash	Отключение планов соединения по хешу с распараллеливанием хеширования. Чтобы использовать планы с распараллеливанием хеширования, также должен быть включён параметр <code>enable_hashjoin</code>.
enable gathermerge	Отключение использования узлов соединения Gather Merge.
debug_parallel_query	Проверка возможности распараллеливания. В версиях PostgreSQL 14 и 15 назывался force_parallel_mode .

- Установка этих параметров в значение **off** не запрещает операции. Планировщик будет пытаться обойтись без них, но может применить в безвыходной ситуации.